

```
In [1]: # Check Python version and verify all required packages are installed
import sys
print('Python version:', sys.version)

# List every package the pipeline needs for extraction, NLP, statistics, output
packages = ['pdfplumber', 'pandas', 'numpy', 'textstat',
            'nltk', 'matplotlib', 'statsmodels', 'openpyxl', 'linearmodels']

# Loop through and flag any that are missing
missing = []
for name in packages:
    try:
        __import__(name)
        print('OK -', name)
    except ImportError:
        print('MISSING -', name)
        missing.append(name)

print()
if missing:
    print('Need to install:', missing)
else:
    print('All packages installed. Ready to go.')
```

Python version: 3.14.6 | packaged by Anaconda, Inc. | (main, Jul 9 2026, 14:29:05) [MSC v.1942 64 bit (AMD64)]

OK - pdfplumber
OK - pandas
OK - numpy
OK - textstat
OK - nltk
OK - matplotlib
OK - statsmodels
OK - openpyxl
OK - linearmodels

All packages installed. Ready to go.

```
In [2]: # Install any missing packages (pdfplumber, textstat, linearmodels)
import subprocess, sys
```

```
subprocess.run([sys.executable, '-m', 'pip', 'install',
                'pdfplumber', 'textstat', 'linearmodels'])
```

Out[2]: CompletedProcess(args=['C:\\ProgramData\\anaconda3\\python.exe', '-m', 'pip', 'install', 'pdfplumber', 'textstat', 'linearmodels'], returncode=0)

In [59]: *# Define climate keywords and the text-extraction function using pdfplumber*

```
import pdfplumber
from pathlib import Path

BASE = Path(r"C:\Users\Sri Ramkrishna\BUSI1783_Dissertation")
REPORTS = BASE / 'annual_reports'

# 18 climate keywords used for line-level filtering
CLIMATE_KEYWORDS = [
    "climate", "carbon", "emission", "greenhouse", "ghg",
    "net zero", "net-zero", "tcf", "decarbon", "renewable",
    "global warming", "paris agreement",
    "scope 1", "scope 2", "scope 3",
    "transition risk", "physical risk", "sustainability"
]

def extract_climate_text(pdf_path):
    """Extract climate-relevant lines from a single PDF annual report."""
    lines_kept = []
    try:
        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                text = page.extract_text()
                if not text:
                    continue
                for line in text.split("\n"):
                    low = line.lower()
                    # Retain the line only if it contains at least one keyword
                    if any(kw in low for kw in CLIMATE_KEYWORDS):
                        words = line.split()
                        # Drop lines shorter than 8 words (headers, labels)
                        if len(words) < 8:
                            continue
                        # Drop lines with digit ratio > 0.30 (tables, figures)
                        digit_ratio = sum(c.isdigit() for c in line) / max(len(line), 1)
```

```

        if digit_ratio > 0.30:
            continue
        lines_kept.append(line.strip())
    except Exception as e:
        print(f" ERROR reading {pdf_path.name}: {e}")
    return " ".join(lines_kept)

# Test on 5 files to verify the extraction works
test_files = ["AAL_2021", "BP_2024", "SHEL_2023", "RIO_2022", "GLEN_2021"]
for name in test_files:
    path = REPORTS / f"{name}.pdf"
    if not path.exists():
        print(f"{name}: FILE NOT FOUND")
        continue
    text = extract_climate_text(path)
    print(f"{name}: {len(text.split())} climate words extracted")

```

AAL_2021: 8737 climate words extracted
 BP_2024: 18451 climate words extracted
 SHEL_2023: 25569 climate words extracted
 RIO_2022: 23712 climate words extracted
 GLEN_2021: 8807 climate words extracted

In [8]: *# Run the full extraction across all 52 firms × 4 years (2021–2024) and save the raw corpus to climate_text.csv*

```

import pandas as pd

companies = pd.read_excel(BASE / 'data' / 'company_list.xlsx')
years = [2021, 2022, 2023, 2024]
records = []
missing = []

for _, firm in companies.iterrows():
    ticker = firm["Ticker"]
    for year in years:
        path = REPORTS / f"{ticker}_{year}.pdf"
        if not path.exists():
            missing.append(f"{ticker}_{year}")
            continue
        text = extract_climate_text(path)
        wc = len(text.split()) if text else 0

```

```
records.append({
    "Ticker": ticker,
    "Year": year,
    "Sector": firm["Sector"],
    "HighCarbon": firm["HighCarbon"],
    "ClimateText": text,
    "WordCount": wc
})
print(f"{ticker}_{year}: {wc} words")

df = pd.DataFrame(records)
print(f"\nMissing files: {len(missing)}")
if missing:
    print(missing)

# Save raw extraction output where this file is the starting point for all scoring
df.to_csv(BASE / 'data' / 'climate_text.csv', index=False)
print(f"\nSaved: {len(df)} rows, {df.Ticker.nunique()} firms")
```

Done: AAL
Done: ANTO
Done: EDV
Done: FRES
Done: GLEN
Done: RIO
Done: BP
Done: SHEL
Done: CNA
Done: NG
Done: SSE
Done: SVT
Done: UU
Done: AHT
Done: BA
Done: BNZL

Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported

Done: DCC
Done: DPLM
Done: EXPN
Done: HLMA
Done: IMI
Done: ITRK
Done: MNDI
Done: MRO
Done: RR
Done: RS1
Done: RTO
Done: SKG
Done: SMDS
Done: SMIN
Done: SPX
Done: WEIR
Done: SGE
Done: BT
Done: VOD

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Done: AZN
Done: GSK
Done: BLND
Done: SGRO
Done: AUTO
Done: NXT

[illegible]

Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 5 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported
 Cannot set non-stroke color: 2 components specified, but only 1 (grayscale), 3 (RGB), and 4 (CMYK) are supported

Done: WPP
 Done: IAG
 Done: PSN
 Done: KGF
 Done: WTB
 Done: ABF
 Done: BATS
 Done: DGE
 Done: RKT
 Done: TSCO
 Done: ULVR

Extracted 208 reports
 Missing files: 0

In [3]: *# Apply all three documented exclusions to produce the definitive clean corpus: BAE Systems (extraction failure), RS*

```
import pandas as pd
from pathlib import Path

BASE = Path(r"C:\Users\Sri Ramkrishna\BUSI1783_Dissertation")

raw = pd.read_csv(BASE / 'data' / 'climate_text.csv')
```

```

print("Input:", len(row), "rows,", row.Ticker.nunique(), "firms")

# Exclusion 1 of BAE Systems: PDF extraction returned no text in 3 of 4 years
raw["ExtractionFailed"] = (raw.WordCount == 0) | raw.ClimateText.isna()
print("\nFailed extractions:")
print(raw[raw.ExtractionFailed][["Ticker", "Year"]])

# Identify firms with incomplete panels (fewer than 3 usable years)
good = raw[~raw.ExtractionFailed].groupby('Ticker').size()
drop_tickers = good[good < 3].index.tolist()
print("\nDropped (incomplete extraction):", drop_tickers)

# Exclusion 2 of RS Group: bullet-point/tabular disclosures, not narrative prose
# Fog and FKGL are sentence-based and undefined for non-prose text
drop_tickers.append("RS1")
print("Dropped (non-prose disclosure): ['RS1']")

# Apply both exclusions
clean = raw[~raw.Ticker.isin(drop_tickers)].reset_index(drop=True)

# Save the definitive clean corpus: 50 firms, 200 firm-year observations
clean.to_csv(BASE / 'data' / 'climate_text_clean.csv', index=False)
print(f"\nOutput: {len(clean)} rows, {clean.Ticker.nunique()} firms")

```

Input: 208 rows, 52 firms

Failed extractions:

	Ticker	Year
56	BA	2021
57	BA	2022
58	BA	2023

Dropped (incomplete extraction): ['BA']

Dropped (non-prose disclosure): ['RS1']

Output: 200 rows, 50 firms

```

In [4]: # Verify the project directory structure and list all data files
from pathlib import Path
BASE = Path(r"C:\Users\Sri Ramkrishna\BUSI1783_Dissertation")
print("BASE =", BASE)

```



```
for f in sorted((BASE / 'data').glob('*')):
    print(' ', f.name)
```

```
BASE = C:\Users\Sri Ramkrishna\BUSI1783_Dissertation
climate_text.csv
climate_text_clean.csv
company_list.xlsx
financial_data.xlsx
Loughran-McDonald_MasterDictionary_1993-2025.csv
master_panel.xlsx
master_panel_winsorized.xlsx
panel_data_enhanced.xlsx
readability.csv
sentiment.csv
```

In [5]: *# Compute readability scores (Fog, FKGL) and flag observations under 100 words for manual review*

```
import pandas as pd
import textstat

df = pd.read_csv(BASE / 'data' / 'climate_text_clean.csv')

# Validate that no observation has empty text
text = df["ClimateText"].fillna("").str.strip()
if (text == "").any():
    raise ValueError(f"Stop: {(text == '').sum()} empty ClimateText rows found.")

# Calculate readability indices where higher value means less readable
df["Fog"] = text.apply(textstat.gunning_fog)
df["FKGL"] = text.apply(textstat.flesch_kincaid_grade)

# Flag observations below the 100-word manual-review threshold from the proposal
df["ManualReview"] = df["WordCount"] < 100

# Save readability dataset
df.to_csv(BASE / 'data' / 'readability.csv', index=False)
print(f"Saved readability.csv: {len(df)} rows, {df.Ticker.nunique()} firms")
print(f"ManualReview flagged: {df.ManualReview.sum()}")
```

Saved readability.csv: 200 rows, 50 firms

ManualReview flagged: 0

```

In [6]: # Score sentiment using the Loughran-McDonald Master Dictionary (1993-2025) to compute positive, negative, and uncertain

import pandas as pd
import re

# Load readability output and LM dictionary
df = pd.read_csv(BASE / 'data' / 'readability.csv')
lm = pd.read_csv(BASE / 'data' / 'Loughran-McDonald_MasterDictionary_1993-2025.csv')

# Standardise dictionary words to uppercase
lm["Word"] = lm["Word"].astype(str).str.upper().str.strip()

# Include only words currently active in each LM category (> 0)
neg_words = set(lm.loc[lm["Negative"] > 0, "Word"])
pos_words = set(lm.loc[lm["Positive"] > 0, "Word"])
unc_words = set(lm.loc[lm["Uncertainty"] > 0, "Word"])
all_lm = set(lm["Word"])

def score_text(text):
    """Tokenise text and count LM-category words."""
    tokens = re.findall(r"[A-Za-z]+", str(text).upper())
    lm_tokens = [t for t in tokens if t in all_lm]
    n_lm = len(lm_tokens)
    n_pos = sum(1 for t in lm_tokens if t in pos_words)
    n_neg = sum(1 for t in lm_tokens if t in neg_words)
    n_unc = sum(1 for t in lm_tokens if t in unc_words)
    return pd.Series({
        "LMWordCount": n_lm,
        "PosCount": n_pos,
        "NegCount": n_neg,
        "UncCount": n_unc,
        # Denominator = LM-matched word count (not total tokens)
        "NetTone": (n_pos - n_neg) / n_lm if n_lm else 0,
        "Uncertainty": n_unc / n_lm if n_lm else 0,
        "Negativity": n_neg / n_lm if n_lm else 0,
    })

# Apply the scoring function to every firm-year's climate text
scores = df["ClimateText"].apply(score_text)
df = pd.concat([df, scores], axis=1)

```



```
# Save sentiment-scored dataset
df.to_csv(BASE / 'data' / 'sentiment.csv', index=False)
print("Sentiment scores saved:", len(df), "rows")
print(df[["Ticker", "Year", "NetTone", "Uncertainty"]].head())
```

Sentiment scores saved: 200 rows

	Ticker	Year	NetTone	Uncertainty
0	AAL	2021	0.003982	0.010753
1	AAL	2022	0.007817	0.013160
2	AAL	2023	0.007932	0.014398
3	AAL	2024	0.004613	0.015634
4	ANTO	2021	0.000158	0.018294

In [7]: *# Build enhanced panel: merge financial data with company list, construct control variables (ROA, Leverage, MtB, Size*

```
import pandas as pd
import numpy as np

fin = pd.read_excel(BASE / 'data' / 'financial_data.xlsx')
comp = pd.read_excel(BASE / 'data' / 'company_list.xlsx')
print("Input:", len(fin), "firm-years,", len(comp), "firms in company list")

# Fix Unicode minus signs and thousand-separator commas (negative values and large figures stored as text by Excel)
for col in ["Net_Income", "Total_Assets", "Total_Debt", "Total_Equity", "Market_Cap"]:
    fin[col] = pd.to_numeric(
        fin[col].astype(str).str.replace("\u2212", "-").str.replace("-", "-").str.replace(",", ""),
        errors="coerce"
    )

# Merge financial data with company metadata (sector, High_Carbon flag)
df = fin.merge(comp[["Ticker", "Sector", "HighCarbon"]], on="Ticker", how="inner")
df.rename(columns={"HighCarbon": "High_Carbon", "Sector": "ICB_Sector"}, inplace=True)

# Construct the four control variables
df["ROA"] = (df.Net_Income / df.Total_Assets).round(4)
df["Leverage"] = (df.Total_Debt / df.Total_Assets).round(4)
df["MtB"] = (df.Market_Cap / df.Total_Equity).round(4)
df["Size"] = np.log(df.Total_Assets).round(4)

# Exclude Smurfit Kappa 2024 because WestRock merger changed entity, GAAP, and currency
df = df[~((df.Ticker == "SKG") & (df.Year == 2024))]
```

```
# Save enhanced panel
df.to_excel(BASE / 'data' / 'panel_data_enhanced.xlsx', index=False)
print(f"Saved: {len(df)} rows, {df.Ticker.nunique()} firms")
```

Input: 204 firm-years, 51 firms in company list

Saved: 203 rows, 51 firms

In [8]: *# Merge NLP scores (readability + sentiment) into the enhanced panel to create the master analysis dataset*

```
import pandas as pd

panel = pd.read_excel(BASE / 'data' / 'panel_data_enhanced.xlsx')
sent = pd.read_csv(BASE / 'data' / 'sentiment.csv')

# Select only the NLP columns needed for the merge
nlp_cols = ["Ticker", "Year", "WordCount", "Fog", "FKGL",
            "LMWordCount", "PosCount", "NegCount", "UncCount",
            "NetTone", "Uncertainty", "Negativity"]

master = panel.merge(sent[nlp_cols], on=["Ticker", "Year"], how="inner")

# Save the master panel (pre-winsorisation)
master.to_excel(BASE / 'data' / 'master_panel.xlsx', index=False)
print("Master panel:", len(master), "rows,", master.Ticker.nunique(), "firms")
```

Master panel: 199 rows, 50 firms

In [9]: *# Winsorise continuous variables at the 1st and 99th percentiles to limit the influence of outliers (e.g. Rolls-Royce*

```
import pandas as pd

master = pd.read_excel(BASE / 'data' / 'master_panel.xlsx')

wins_vars = ["ROA", "Leverage", "MtB", "Fog", "FKGL",
            "NetTone", "Uncertainty", "Negativity"]

print("Winsorising at 1st/99th percentile:\n")
for v in wins_vars:
    p1, p99 = master[v].quantile(0.01), master[v].quantile(0.99)
    n = ((master[v] < p1) | (master[v] > p99)).sum()
    master[v] = master[v].clip(p1, p99)
    print(f" {v:<12} {n:>2} values capped to [{p1:.4f}, {p99:.4f}]" )
```



```
# Save the winsorised master panel: this is the final analysis dataset
master.to_excel(BASE / 'data' / 'master_panel_winsorized.xlsx', index=False)
print(f"\nSaved: {len(master)} rows, {master.Ticker.nunique()} firms")
```

Winsorising at 1st/99th percentile:

```
ROA          4 values capped to [-0.1221, 0.3537]
Leverage      4 values capped to [0.0092, 0.5991]
MtB           4 values capped to [-2.3086, 12.8716]
Fog           4 values capped to [18.0625, 27.1143]
FKGL          4 values capped to [14.9272, 23.6147]
NetTone       4 values capped to [-0.0032, 0.0229]
Uncertainty   4 values capped to [0.0069, 0.0289]
Negativity    4 values capped to [0.0038, 0.0141]
```

Saved: 199 rows, 50 firms

In [10]: *# Produce descriptive statistics (Table 4.1) and univariate high- vs low-carbon comparison with t-tests (Table 4.2)*

```
import pandas as pd
from scipy import stats

master = pd.read_excel(BASE / 'data' / 'master_panel_winsorized.xlsx')

desc_vars = ["Fog", "FKGL", "NetTone", "Uncertainty", "Negativity",
             "WordCount", "ROA", "Leverage", "MtB", "Size"]

# Table 4.1: descriptive statistics for the full sample
print("TABLE 4.1: DESCRIPTIVE STATISTICS (N = 199)")
print("=" * 80)
desc = master[desc_vars].describe().T[["mean", "std", "min", "25%", "50%", "75%", "max"]]
print(desc.round(4).to_string())

# Table 4.2: compare high-carbon vs low-carbon group means
print("\n\nTABLE 4.2: HIGH-CARBON vs LOW-CARBON COMPARISON")
print("=" * 80)
compare_vars = ["Fog", "FKGL", "NetTone", "Uncertainty", "WordCount"]
hi = master[master.High_Carbon == 1]
lo = master[master.High_Carbon == 0]

print(f"  High-carbon: {len(hi)} obs ({hi.Ticker.nunique()} firms)")
print(f"  Low-carbon:  {len(lo)} obs ({lo.Ticker.nunique()} firms)\n")
```

```
# Equal-variance t-tests (matching the coding style from seminars)
print(f"{'Variable':>12} {'High-C':>8} {'Low-C':>8} {'Diff':>8} {'t':>7} {'p':>7}")
print("-" * 62)
for v in compare_vars:
    h, l = hi[v], lo[v]
    t_stat, p_val = stats.ttest_ind(h, l, equal_var=True)
    diff = h.mean() - l.mean()
    star = "***" if p_val < 0.01 else "*" if p_val < 0.05 else "" if p_val < 0.1 else ""
    print(f"{'v':>12} {h.mean():8.4f} {l.mean():8.4f} {diff:+8.4f} {t_stat:7.3f} {p_val:7.4f}{star}")
```

TABLE 4.1: DESCRIPTIVE STATISTICS (N = 199)

	mean	std	min	25%	50%	75%	max
Fog	21.4791	1.7662	18.0625	20.3134	21.3791	22.3755	27.1143
FKGL	18.1023	1.6909	14.9272	17.0061	17.9934	18.9243	23.6147
NetTone	0.0070	0.0045	-0.0032	0.0040	0.0067	0.0091	0.0229
Uncertainty	0.0163	0.0047	0.0069	0.0131	0.0162	0.0193	0.0289
Negativity	0.0083	0.0022	0.0038	0.0069	0.0080	0.0097	0.0141
WordCount	8301.3970	5940.2403	1014.0000	4638.5000	6764.0000	9964.0000	41198.0000
ROA	0.0595	0.0696	-0.1221	0.0258	0.0567	0.0867	0.3537
Leverage	0.2847	0.1309	0.0092	0.2024	0.2805	0.3565	0.5991
MtB	3.3338	2.9756	-2.3086	1.2881	2.1079	4.9670	12.8716
Size	9.9093	1.3587	6.5699	9.0588	9.7572	11.0300	13.0014

TABLE 4.2: HIGH-CARBON vs LOW-CARBON COMPARISON

High-carbon: 119 obs (30 firms)					
Low-carbon: 80 obs (20 firms)					
Variable	High-C	Low-C	Diff	t	p
Fog	21.7079	21.1386	+0.5693	2.252	0.0254**
FKGL	18.2274	17.9163	+0.3111	1.274	0.2040
NetTone	0.0073	0.0065	+0.0007	1.115	0.2662
Uncertainty	0.0157	0.0172	-0.0015	-2.219	0.0276**
WordCount	9689.7563	6236.2125	+3453.5438	4.186	0.0000***

In [11]: # Compute the correlation matrix (Figure 4.2) and VIF diagnostics

```
import pandas as pd
```



```

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

master = pd.read_excel(BASE / 'data' / 'master_panel_winsorized.xlsx')

corr_vars = ["Fog", "FKGL", "NetTone", "Uncertainty",
             "High_Carbon", "ROA", "Size", "Leverage", "MtB"]

# Correlation matrix
print("CORRELATION MATRIX (Figure 4.2)")
print("=" * 80)
print(master[corr_vars].corr().round(3).to_string())

# VIF to check for multicollinearity among explanatory variables
print("\n\nVIF DIAGNOSTICS (multicollinearity check)")
print("-" * 40)
X = master[["High_Carbon", "ROA", "Size", "Leverage", "MtB"]].assign(const=1)
for i, col in enumerate(X.columns):
    if col != "const":
        print(f" {col:<14} VIF = {variance_inflation_factor(X.values, i):.3f}")

```

CORRELATION MATRIX (Figure 4.2)

```

=====

```

	Fog	FKGL	NetTone	Uncertainty	High_Carbon	ROA	Size	Leverage	MtB
Fog	1.000	0.981	0.136	0.024	0.158	-0.165	0.007	-0.001	-0.058
FKGL	0.981	1.000	0.171	-0.014	0.090	-0.177	-0.013	0.038	-0.027
NetTone	0.136	0.171	1.000	-0.117	0.079	-0.062	-0.407	0.008	0.076
Uncertainty	0.024	-0.014	-0.117	1.000	-0.156	0.036	-0.135	-0.058	-0.034
High_Carbon	0.158	0.090	0.079	-0.156	1.000	-0.026	-0.173	-0.076	-0.112
ROA	-0.165	-0.177	-0.062	0.036	-0.026	1.000	-0.279	-0.207	0.463
Size	0.007	-0.013	-0.407	-0.135	-0.173	-0.279	1.000	0.147	-0.458
Leverage	-0.001	0.038	0.008	-0.058	-0.076	-0.207	0.147	1.000	0.214
MtB	-0.058	-0.027	0.076	-0.034	-0.112	0.463	-0.458	0.214	1.000

VIF DIAGNOSTICS (multicollinearity check)

```

-----
High_Carbon    VIF = 1.083
ROA            VIF = 1.455
Size           VIF = 1.465
Leverage       VIF = 1.292
MtB            VIF = 1.903

```

```

In [12]: # Run pooled OLS regressions with year fixed effects and firm-clustered SEs (Table 4.3: main results)

import pandas as pd
import statsmodels.api as sm

master = pd.read_excel(BASE / 'data' / 'master_panel_winsorized.xlsx')

# Create year dummies (2021 is the omitted base category)
master["Y2022"] = (master.Year == 2022).astype(int)
master["Y2023"] = (master.Year == 2023).astype(int)
master["Y2024"] = (master.Year == 2024).astype(int)

ivs = ["High_Carbon", "ROA", "Size", "Leverage", "MtB",
       "Y2022", "Y2023", "Y2024"]
dvs = ["Fog", "FKGL", "NetTone", "Uncertainty"]

# Estimate one model per dependent variable
res = {}
for dv in dvs:
    X = sm.add_constant(master[ivs].astype(float))
    y = master[dv]
    model = sm.OLS(y, X).fit(cov_type="cluster",
                             cov_kws={"groups": master["Ticker"]})
    res[dv] = model

# Print formatted regression table
def star(p):
    return "***" if p < 0.01 else "*" if p < 0.05 else "" if p < 0.1 else ""

print("TABLE 4.3: POOLED OLS, YEAR FIXED EFFECTS, FIRM-CLUSTERED SEs")
print("=" * 80)
print(f"{'Variable':<14}" + ''.join(f"{d:>16}" for d in dvs))
print("-" * 78)

for iv in ivs:
    coef_line = f"{iv:<14}"
    se_line = f"{'':14}"
    for d in dvs:
        c = res[d].params[iv]
        p = res[d].pvalues[iv]
        s = res[d].bse[iv]

```



```

        coef_line += f"{c:>12.4f}{star(p):<4}"
        se_line   += f"'(' + f'{s:.4f}' + ')':>16}"
    print(coef_line)
    print(se_line)

print(f"\n{'N':<14}" + ''.join(f"{int(res[d].nobs):>16}" for d in dvs))
print(f"{'R-squared':<14}" + ''.join(f"{res[d].rsquared:>16.4f}" for d in dvs))
print("\n*** p<0.01, ** p<0.05, * p<0.1")

```

TABLE 4.3: POOLED OLS, YEAR FIXED EFFECTS, FIRM-CLUSTERED SEs

Variable	Fog	FKGL	NetTone	Uncertainty
High_Carbon	0.6047 (0.4276)	0.3261 (0.4152)	-0.0002 (0.0007)	-0.0020** (0.0010)
ROA	-5.3104* (2.9740)	-5.6363* (2.9215)	-0.0094 (0.0058)	0.0033 (0.0075)
Size	0.0478 (0.1616)	-0.0080 (0.1574)	-0.0017*** (0.0004)	-0.0008** (0.0004)
Leverage	-0.7602 (1.4936)	-0.2943 (1.4732)	0.0026 (0.0029)	0.0003 (0.0043)
MtB	0.0672 (0.0753)	0.0655 (0.0729)	-0.0002 (0.0002)	-0.0003 (0.0002)
Y2022	0.7062*** (0.2067)	0.6122*** (0.2023)	0.0003 (0.0006)	0.0019*** (0.0007)
Y2023	0.6367** (0.2494)	0.4800** (0.2361)	-0.0005 (0.0007)	0.0018** (0.0008)
Y2024	0.8046*** (0.2886)	0.5895** (0.2698)	-0.0010 (0.0008)	0.0030*** (0.0009)
N	199	199	199	199
R-squared	0.0855	0.0656	0.2183	0.1283

*** p<0.01, ** p<0.05, * p<0.1

In [13]: # Appendix C: Full regression summaries (intercepts + exact p-values)

```

print("APPENDIX C: FULL REGRESSION OUTPUT")
print("=" * 80)

for dv in dvs:
    print(f"\n{'='*80}")

```

```
print(f"Dependent variable: {dv}")  
print(f"{'='*80}")  
print(res[dv].summary())
```


APPENDIX C: FULL REGRESSION OUTPUT

Dependent variable: Fog

OLS Regression Results

```

=====
Dep. Variable:          Fog    R-squared:                0.086
Model:                  OLS    Adj. R-squared:           0.047
Method:                 Least Squares    F-statistic:           2.349
Date:                  Wed, 26 Aug 2026    Prob (F-statistic):     0.0318
Time:                  15:39:09    Log-Likelihood:        -386.18
No. Observations:      199    AIC:                   790.4
Df Residuals:          190    BIC:                   820.0
Df Model:               8
Covariance Type:       cluster
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	20.4166	1.762	11.588	0.000	16.964	23.870
High_Carbon	0.6047	0.428	1.414	0.157	-0.233	1.443
ROA	-5.3104	2.974	-1.786	0.074	-11.139	0.518
Size	0.0478	0.162	0.296	0.767	-0.269	0.365
Leverage	-0.7602	1.494	-0.509	0.611	-3.688	2.167
MtB	0.0672	0.075	0.892	0.372	-0.080	0.215
Y2022	0.7062	0.207	3.417	0.001	0.301	1.111
Y2023	0.6367	0.249	2.553	0.011	0.148	1.126
Y2024	0.8046	0.289	2.788	0.005	0.239	1.370

```

=====
Omnibus:                35.108    Durbin-Watson:           0.930
Prob(Omnibus):           0.000    Jarque-Bera (JB):        55.718
Skew:                    0.957    Prob(JB):                 7.96e-13
Kurtosis:                4.747    Cond. No.                 189.
=====

```

Notes:

[1] Standard Errors are robust to cluster correlation (cluster)

Dependent variable: FKGL

OLS Regression Results

```

=====
Dep. Variable:          FKGL    R-squared:          0.066
Model:                  OLS     Adj. R-squared:       0.026
Method:                 Least Squares    F-statistic:       2.235
Date:                   Wed, 26 Aug 2026    Prob (F-statistic): 0.0405
Time:                   15:39:09    Log-Likelihood:    -379.65
No. Observations:      199    AIC:              777.3
Df Residuals:          190    BIC:              806.9
Df Model:               8
Covariance Type:       cluster
=====

```

```

=====
              coef    std err          z      P>|z|      [0.025      0.975]
-----
const          17.7679      1.689     10.521     0.000      14.458      21.078
High_Carbon      0.3261      0.415      0.785     0.432      -0.488       1.140
ROA             -5.6363      2.922     -1.929     0.054     -11.362       0.090
Size            -0.0080      0.157     -0.051     0.959     -0.316       0.300
Leverage        -0.2943      1.473     -0.200     0.842     -3.182       2.593
MtB              0.0655      0.073      0.899     0.369     -0.077       0.208
Y2022            0.6122      0.202      3.026     0.002       0.216       1.009
Y2023            0.4800      0.236      2.033     0.042       0.017       0.943
Y2024            0.5895      0.270      2.185     0.029       0.061       1.118
=====

```

```

=====
Omnibus:          42.109    Durbin-Watson:      0.917
Prob(Omnibus):    0.000    Jarque-Bera (JB):    74.834
Skew:             1.071    Prob(JB):            5.62e-17
Kurtosis:         5.106    Cond. No.            189.
=====

```

Notes:

[1] Standard Errors are robust to cluster correlation (cluster)

```

=====
Dependent variable: NetTone
=====

```

OLS Regression Results

```

=====
Dep. Variable:          NetTone    R-squared:          0.218
Model:                  OLS     Adj. R-squared:       0.185
Method:                 Least Squares    F-statistic:       5.959
Date:                   Wed, 26 Aug 2026    Prob (F-statistic): 2.47e-05

```

Time: 15:39:09 Log-Likelihood: 819.47
 No. Observations: 199 AIC: -1621.
 Df Residuals: 190 BIC: -1591.
 Df Model: 8
 Covariance Type: cluster

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----+-----
const          0.0245      0.004       5.525      0.000       0.016      0.033
High_Carbon   -0.0002      0.001      -0.261      0.794     -0.002      0.001
ROA            -0.0094      0.006     -1.608      0.108     -0.021      0.002
Size           -0.0017      0.000     -4.326      0.000     -0.002     -0.001
Leverage       0.0026      0.003       0.895      0.371     -0.003      0.008
MtB            -0.0002      0.000     -1.092      0.275     -0.000      0.000
Y2022          0.0003      0.001       0.429      0.668     -0.001      0.002
Y2023          -0.0005      0.001     -0.732      0.464     -0.002      0.001
Y2024          -0.0010      0.001     -1.290      0.197     -0.002      0.001
=====
```

```
=====
Omnibus:          24.967   Durbin-Watson:          1.139
Prob(Omnibus):    0.000   Jarque-Bera (JB):        39.270
Skew:             0.704   Prob(JB):                 2.97e-09
Kurtosis:         4.659   Cond. No.                  189.
=====
```

Notes:

[1] Standard Errors are robust to cluster correlation (cluster)

```
=====
Dependent variable: Uncertainty
=====
```

OLS Regression Results

```
=====
Dep. Variable:      Uncertainty   R-squared:          0.128
Model:              OLS          Adj. R-squared:      0.092
Method:             Least Squares  F-statistic:        3.785
Date:               Wed, 26 Aug 2026  Prob (F-statistic):  0.00159
Time:               15:39:09      Log-Likelihood:     799.27
No. Observations:   199          AIC:                 -1581.
Df Residuals:       190          BIC:                 -1551.
Df Model:           8
Covariance Type:    cluster
=====
```


	coef	std err	z	P> z	[0.025	0.975]
const	0.0244	0.004	5.707	0.000	0.016	0.033
High_Carbon	-0.0020	0.001	-1.976	0.048	-0.004	-1.62e-05
ROA	0.0033	0.007	0.447	0.655	-0.011	0.018
Size	-0.0008	0.000	-2.013	0.044	-0.002	-2.13e-05
Leverage	0.0003	0.004	0.078	0.938	-0.008	0.009
MtB	-0.0003	0.000	-1.560	0.119	-0.001	6.42e-05
Y2022	0.0019	0.001	2.639	0.008	0.000	0.003
Y2023	0.0018	0.001	2.346	0.019	0.000	0.003
Y2024	0.0030	0.001	3.371	0.001	0.001	0.005
=====						
Omnibus:		1.216	Durbin-Watson:			1.175
Prob(Omnibus):		0.545	Jarque-Bera (JB):			1.171
Skew:		0.186	Prob(JB):			0.557
Kurtosis:		2.947	Cond. No.			189.
=====						

Notes:

[1] Standard Errors are robust to cluster correlation (cluster)

In [14]: *# Hausman test: compare Random Effects vs Fixed Effects for each DV to confirm whether the pooled specification is ap*

```
import pandas as pd
import numpy as np
from statsmodels.tools import add_constant
from linearmodels.panel import PanelOLS, RandomEffects
from scipy import stats
import warnings; warnings.filterwarnings('ignore')

master = pd.read_excel(BASE / 'data' / 'master_panel_winsorized.xlsx')

# Create year dummies for the panel estimators
for y in [2022, 2023, 2024]:
    master[f"Y{y}"] = (master.Year == y).astype(float)

# Set the panel index (Ticker x Year)
p = master.set_index(["Ticker", "Year"])

tv = ["ROA", "Size", "Leverage", "MtB", "Y2022", "Y2023", "Y2024"]

# Run Hausman test for each dependent variable
```

```

for dv in ["Fog", "FKGL", "NetTone", "Uncertainty"]:
    y = p[dv]
    X = add_constant(p[tv].astype(float))

    # Estimate FE and RE models
    fe = PanelOLS(y, X, entity_effects=True).fit()
    re = RandomEffects(y, X).fit()

    # Hausman statistic: (b_FE - b_RE)' [Var(b_FE) - Var(b_RE)]^-1 (b_FE - b_RE)
    b_diff = fe.params[tv] - re.params[tv]
    v_diff = fe.cov[tv].loc[tv] - re.cov[tv].loc[tv]
    chi2 = float(b_diff @ np.linalg.pinv(v_diff) @ b_diff)
    df = len(tv)
    p_val = 1 - stats.chi2.cdf(chi2, df)
    verdict = "FE preferred" if p_val < 0.05 else "RE preferred"

    print(f"{dv:<12} Hausman chi2 = {chi2:8.2f}    df = {df}    "
          f"p = {p_val:.3f}    → {verdict}")

```

Fog	Hausman chi2 =	9.65	df = 7	p = 0.209	→ RE preferred
FKGL	Hausman chi2 =	9.77	df = 7	p = 0.202	→ RE preferred
NetTone	Hausman chi2 =	0.85	df = 7	p = 0.997	→ RE preferred
Uncertainty	Hausman chi2 =	5.65	df = 7	p = 0.581	→ RE preferred

In [15]: *# Robustness checks: re-run pooled OLS excluding Energy, excluding Rolls-Royce, and reclassifying IAG as high-carbon*

```

import pandas as pd
import statsmodels.api as sm

ivs = ["High_Carbon", "ROA", "Size", "Leverage", "MtB",
        "Y2022", "Y2023", "Y2024"]
dvs = ["Fog", "FKGL", "NetTone", "Uncertainty"]

def run(df, label):
    """Run pooled OLS with year FE and firm-clustered SE on a given subset."""
    for y in [2022, 2023, 2024]:
        df[f"Y{y}"] = (df.Year == y).astype(int)

    print(f"\n{label} (N = {len(df)}, firms = {df.Ticker.nunique()})")
    print("-" * 62)
    for dv in dvs:
        X = sm.add_constant(df[ivs].astype(float))

```

```

m = sm.OLS(df[dv], X).fit(cov_type="cluster",
                        cov_kwds={"groups": df["Ticker"]})
hc = m.params["High_Carbon"]
p = m.pvalues["High_Carbon"]
s = "****" if p < 0.01 else "***" if p < 0.05 else "*" if p < 0.1 else ""
print(f" {dv:<12} High_Carbon = {hc:+.4f}{s:<4} "
      f"(SE {m.bse['High_Carbon']:.4f}, p = {p:.3f})")

# (a) Baseline: full winsorised sample
run(master.copy(), "BASELINE (winsorised)")

# (b) Exclude Energy sector
run(master[master.ICB_Sector != "Energy"].copy(), "EXCLUDE ENERGY")

# (c) Exclude Rolls-Royce (negative equity → negative MtB)
run(master[master.Ticker != "RR"].copy(), "EXCLUDE ROLLS-ROYCE")

# (d) Reclassify IAG as high-carbon (aviation emissions intensity)
iag = master.copy()
iag.loc[iag.Ticker == "IAG", "High_Carbon"] = 1
run(iag, "RECLASSIFY IAG AS HIGH-CARBON")

```


BASELINE (winsorised) (N = 199, firms = 50)

```
-----
Fog          High_Carbon = +0.6047      (SE 0.4276, p = 0.157)
FKGL         High_Carbon = +0.3261      (SE 0.4152, p = 0.432)
NetTone      High_Carbon = -0.0002      (SE 0.0007, p = 0.794)
Uncertainty  High_Carbon = -0.0020**    (SE 0.0010, p = 0.048)
```

EXCLUDE ENERGY (N = 191, firms = 48)

```
-----
Fog          High_Carbon = +0.6484      (SE 0.4386, p = 0.139)
FKGL         High_Carbon = +0.4232      (SE 0.4193, p = 0.313)
NetTone      High_Carbon = -0.0001      (SE 0.0008, p = 0.922)
Uncertainty  High_Carbon = -0.0024**    (SE 0.0011, p = 0.027)
```

EXCLUDE ROLLS-ROYCE (N = 195, firms = 49)

```
-----
Fog          High_Carbon = +0.6020      (SE 0.4282, p = 0.160)
FKGL         High_Carbon = +0.3175      (SE 0.4155, p = 0.445)
NetTone      High_Carbon = -0.0001      (SE 0.0007, p = 0.849)
Uncertainty  High_Carbon = -0.0021**    (SE 0.0010, p = 0.040)
```

RECLASSIFY IAG AS HIGH-CARBON (N = 199, firms = 50)

```
-----
Fog          High_Carbon = +0.7792*     (SE 0.4145, p = 0.060)
FKGL         High_Carbon = +0.4909      (SE 0.4050, p = 0.225)
NetTone      High_Carbon = +0.0001      (SE 0.0007, p = 0.928)
Uncertainty  High_Carbon = -0.0026**    (SE 0.0010, p = 0.010)
```

In [16]: *# Generate all figures: correlation heatmap, high-vs-low bar charts, and readability trend over time*

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

master = pd.read_excel(BASE / 'data' / 'master_panel_winsorized.xlsx')
FIG = BASE / 'figures'
FIG.mkdir(exist_ok=True)

# Figure 4.1: high-carbon vs Low-carbon bar charts by year
metrics = ["Fog", "FKGL", "NetTone", "Uncertainty"]
titles  = ["Gunning Fog Index", "Flesch-Kincaid Grade Level",
```

```

        "LM Net Tone", "LM Uncertainty Ratio"]
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Climate Disclosure Metrics: High- vs Low-Carbon Firms, 2021-2024",
            fontsize=13)
for ax, metric, title in zip(axes.flat, metrics, titles):
    grp = master.groupby(["Year", "High_Carbon"])[metric].mean().unstack()
    grp.columns = ["Low-carbon", "High-carbon"]
    grp[["High-carbon", "Low-carbon"]].plot(kind="bar", ax=ax,
                                           color=["#d62728", "#1f77b4"])

    ax.set_title(title)
    ax.set_xlabel("")
    ax.legend(fontsize=8)
plt.tight_layout()
plt.savefig(FIG / 'fig4_1_highcarbon_vs_lowcarbon.png', dpi=150)
plt.close()
print("Saved: fig4_1_highcarbon_vs_lowcarbon.png")

# Figure 4.2: correlation matrix heatmap
cv = ["Fog", "FKGL", "NetTone", "Uncertainty",
      "High_Carbon", "ROA", "Size", "Leverage", "MtB"]
c = master[cv].corr()
fig, ax = plt.subplots(figsize=(9, 7.5))
im = ax.imshow(c, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(cv))); ax.set_yticks(range(len(cv)))
ax.set_xticklabels(cv, rotation=45, ha="right", fontsize=9)
ax.set_yticklabels(cv, fontsize=9)
for i in range(len(cv)):
    for j in range(len(cv)):
        ax.text(j, i, f"{c.iloc[i, j]:.2f}", ha="center", va="center", fontsize=8)
plt.colorbar(im, ax=ax)
ax.set_title("Correlation Matrix")
plt.tight_layout()
plt.savefig(FIG / 'fig4_2_correlation_matrix.png', dpi=150)
plt.close()
print("Saved: fig4_2_correlation_matrix.png")

# Readability trend over time (whole sample)
yearly = master.groupby("Year")[["Fog", "FKGL"]].mean()
fig, ax = plt.subplots(figsize=(7, 4))
yearly.plot(ax=ax, marker="o")
ax.set_title("Average Readability Scores, 2021-2024")
ax.set_ylabel("Index value")

```

```
plt.tight_layout()  
plt.savefig(FIG / 'fig_readability_trend.png', dpi=150)  
plt.close()  
print("Saved: fig_readability_trend.png")
```

Saved: fig4_1_highcarbon_vs_lowcarbon.png

Saved: fig4_2_correlation_matrix.png

Saved: fig_readability_trend.png